

山东大学



实验一实验报告 Experiment report

组员 1：熊梓刚、组员 2：高焜、组员 3：马旭恩、组员 4：陈奕顺

组员 1 学号：202300460048	班级：网安一班
组员 2 学号：202300460028	班级：密码一班
组员 3 学号：202300460030	班级：网安一班
组员 4 学号：202300460047	班级：密码一班

声明

本实验主要由熊梓刚、高焜、马旭恩完成。

马旭恩完成的部分：比特币与区块链基础知识调研，撰写实验报告，完成区块头解析及难度目标计算。

高焜完成的部分：交易数据结构的深入分析，撰写实验报告，完成脚本解码与 UTXO 模型分析。

熊梓刚完成的部分：项目代码实现与自动化验证，撰写实验报告，完成 TxID、Block Hash 和 Merkle Root 的验证。

2026 年 7 月 20 日

1 比特币与区块链简介

1.1 什么是比特币

比特币是由中本聪于 2008 年提出的去中心化数字货币系统。其核心创新在于利用密码学原理和点对点网络，构建了一个无需可信第三方的电子支付系统。所有的交易记录被公开、不可篡改地存储在一个被称为”区块链”的分布式账本中。

1.2 区块链

区块链本质上是一个**单向链表**，每个区块通过包含前一个区块的哈希值来链接在一起。任何一个区块的数据被修改，将导致其哈希值变化，进而使得后续所有区块的哈希值不匹配。这种**链式结构**和**工作量证明 (PoW)** 机制共同保证了区块链的不可篡改性。

区块链的核心特性

- **去中心化**：没有中央服务器，所有节点平等参与。
- **不可篡改**：已确认的区块无法被修改。
- **透明性**：所有交易公开可见。
- **无需信任**：密码学验证代替了对第三方的信任。

1.3 UTXO 模型

Bitcoin 采用**未花费交易输出 (UTXO, 也就是 Unspent Transaction Output)** 模型。每一笔交易的输入引用之前某笔交易的输出，并在花费后将其标记为已花费。一个比特币地址的余额即为其所有未花费 UTXO 的面值之和。这与传统的账户/余额模型有本质区别。

1.4 本实验概述

本实验在 Bitcoin Testnet 上完成了一笔真实交易，并对该交易的原始字节以及所在区块进行了逐字段解析。所有解析工作均使用 Python 从字节层面手动完成，不使用 Bitcoin 库进行解析。

2 比特币区块的数据结构

2.1 区块的整体结构

一个 Bitcoin Block 主要由两大部分组成：

1. **区块头**：固定 80 字节，包含区块的元数据。
2. **交易列表**：可变长度，包含该区块内的所有交易。

区块序列化格式

字段	大小	描述
Block Header	80 bytes	区块头（内含 Version、PrevHash、Merkle Root、Timestamp、Bits、Nonce）
Tx Count	varint	交易数量（其使用 CompactSize 编码）
Transactions	variable	序列化的交易列表

2.2 序列化说明

Bitcoin 使用**小端序**编码所有整数字段。哈希值在内部存储时也使用小端序，但在对外展示时通常转换为大端序。这两种表示之间需要**逐字节反转**。

CompactSize 整数

比特币使用可变长度整数来编码数量、长度等字段，以节省空间：

值范围	编码格式	字节数
$< 0xFD$	uint8_t	1
$\leq 0xFFFF$	0xFD + uint16_t (LE)	3
$\leq 0xFFFFFFFF$	0xFE + uint32_t (LE)	5
$> 0xFFFFFFFF$	0xFF + uint64_t (LE)	9

3 区块头的结构

3.1 区块头的 80 字节结构

区块头是区块中最重要的部分，包含了 PoW 挖矿所需的所有参数。矿工通过不断尝试不同的 Nonce 值，使得区块头的双 SHA-256 哈希值小于目标值。

字段	大小	类型	描述
Version	4 bytes	int32_t (LE)	区块版本号
Previous Block Hash	32 bytes	char[32]	前一区块的哈希（内部小端序）
Merkle Root	32 bytes	char[32]	本区块所有交易的 Merkle 树根
Timestamp	4 bytes	uint32_t (LE)	Unix 时间戳（秒）
Bits (nBits)	4 bytes	uint32_t (LE)	紧凑格式的难度目标
Nonce	4 bytes	uint32_t (LE)	PoW 随机数

3.2 各字段详解

Version（版本号）

标识区块的版本。Bitcoin 通过版本号来支持协议升级。例如 BIP-9 使用 version bits 来信号支持新的共识规则。

Previous Block Hash (前一区块哈希)

前一个区块的 Block Hash (即前一个区块头的双 SHA-256 哈希值)。正是这个字段使区块形成了“链”。任何对前驱区块的修改都会改变此哈希,从而使本区块的内容不再有效。

Merkle Root (梅克尔根)

本区块中所有交易的 Merkle 树的根哈希。Merkle 树是一种二叉树,叶子节点是每笔交易的 TxID,内部节点是其子节点拼接后的双 SHA-256 哈希。Merkle Root 使得轻节点 (SPV) 可以验证某笔交易是否包含在区块中而无需下载整个区块。

Timestamp (时间戳)

区块被创建时的 Unix 时间戳 (秒)。Bitcoin 协议要求时间戳大于前 11 个区块的中位数时间,且小于网络调整时间 + 2 小时。

Bits (难度目标)

紧凑格式的难度目标值。第一个字节是指数,后三个字节是系数:

$$\text{Target} = \text{coefficient} \times 2^{8 \times (\text{exponent} - 3)}$$

Nonce (随机数)

矿工不断尝试不同的 Nonce 值,直到找到满足以下条件的 Block Hash:

$$\text{SHA256}(\text{SHA256}(\text{BlockHeader})) < \text{Target}$$

3.3 区块哈希计算

区块哈希是区块头的双 SHA-256 哈希值,下面以 RPC 字节序显示:

$$\text{BlockHash} = \text{rev}(\text{SHA256}(\text{SHA256}(\text{80-byte header})))$$

4 Transaction 的数据结构

4.1 交易的序列化格式

Bitcoin 交易描述了一笔或多笔 UTXO 的转移过程。一笔交易包含输入列表和输出列表:

字段	大小	描述
Version	4 bytes	交易版本号
<i>[Marker]</i>	1 byte (opt.)	SegWit 标记 (0x00)
<i>[Flag]</i>	1 byte (opt.)	SegWit 标志 (0x01)
Input Count	varint	输入数量
Inputs	variable	输入列表
Output Count	varint	输出数量
Outputs	variable	输出列表
<i>[Witness]</i>	variable (opt.)	见证数据 (仅 SegWit 交易)
Locktime	4 bytes	锁定时间

4.2 SegWit 交易识别

如果 Version 之后的第一个字节是 0x00、第二个字节是 0x01，则该交易为 SegWit 交易。此时 Input Count 从第 6 个字节开始，并且在所有 Output 之后、Locktime 之前会包含 Witness 数据。

4.3 各字段详解

Version (版本号)

交易格式的版本。当前常见的版本号为 1 (非 SegWit) 或 2 (支持 BIP-68 相对锁定时间)。SegWit 交易的版本号通常也为 1，但通过在 Version 之后添加 Marker 和 Flag 来区分。

Input Count (输入数量)

该交易消耗的 UTXO 数量。使用 CompactSize 编码。

Output Count (输出数量)

该交易创建的 UTXO 数量。使用 CompactSize 编码。

Locktime (锁定时间)

交易的锁定时间：

- 如果 $< 500,000,000$ ：表示区块高度，且该区块之前交易无效。
- 如果 $\geq 500,000,000$ ：表示 Unix 时间戳。
- 如果所有输入的 Sequence 都是 0xFFFFFFFF，Locktime 被禁用。

4.4 TxID 计算

交易的 TxID 是交易数据，其是不包括 Witness 数据的双 SHA-256 哈希：

$$\text{TxID} = \text{rev}(\text{SHA256}(\text{SHA256}(\text{raw_tx_without_witness})))$$

需要注意的是，SegWit 交易的 TxID 计算**不包含** Witness 数据，这解决了交易可塑性 (Transaction Malleability) 问题。

5 交易输入

5.1 Input 的结构

每个 Transaction Input 引用一个之前存在的 UTXO，并提供“解锁”该 UTXO 所需的 scriptSig。

字段	大小	描述
Previous Tx Hash	32 bytes	被引用的交易的 TxID（内部字节序）
Previous Output Index	4 bytes	被引用的输出在交易中的索引（从 0 开始）
scriptSig Length	varint	scriptSig 的字节长度
scriptSig	variable	解锁脚本（签名 + 公钥）
Sequence	4 bytes	序列号（用于交易替换和相对锁定时间）

5.2 各字段详解

Previous Tx Hash（前置交易哈希）

指向被花费的 UTXO 所在的交易。该哈希在区块链内部存储为**小端序**，在解析时需要反转回大端序才能得到人类可读的 TxID。

Previous Output Index（前置输出索引）

指定被花费的是前置交易的第几个输出。一个 TxID + Output Index 的组合唯一标识了一个 UTXO，称为 OutPoint。

scriptSig（解锁脚本）

用于满足前置输出中 scriptPubKey 花费条件的脚本。对于最常见的 P2PKH（Pay-to-Public-Key-Hash）输出，scriptSig 包含两个元素：

1. **签名 (Signature)**：使用私钥对交易摘要的 ECDSA 签名（DER 编码）。
2. **公钥 (Public Key)**：对应的 secp256k1 公钥（压缩或未压缩）。

Sequence（序列号）

最初设计用于交易替换，但后来被 BIP-68 扩展为**相对锁定时间 (Relative Locktime)** 的支持字段。当 Sequence 为 0xFFFFFFFF 时，表示该输入是“最终”的，不允许替换且不启用锁定时间。

5.3 scriptSig 的典型格式

对于 P2PKH 输入（最常见的类型），scriptSig 的字节布局为：

1 <sig_len> <sig_der_bytes> <pubkey_len> <pubkey_bytes>

其中：

- sig_len 是签名长度（1 字节，通常为 0x47–0x48，即 71–72 字节）。
- sig_der_bytes 是 DER 编码的 ECDSA 签名，末尾附加 1 字节的 SIGHASH 类型。

- `pubkey_len` 是公钥长度 (1 字节, 压缩公钥为 0x21=33 字节)。
- `pubkey_bytes` 是 secp256k1 公钥 (压缩格式以 02 或 03 开头)。

6 交易输出

6.1 Output 的结构

每个 Transaction Output 创建一个新的 UTXO, 包含金额和花费条件。

字段	大小	描述
Amount	8 bytes	金额 (单位: 聪 satoshi, LE uint64)
scriptPubKey Length	varint	scriptPubKey 的字节长度
scriptPubKey	variable	锁定脚本 (花费条件)

6.2 各字段详解

Amount (金额)

以聪 (satoshi) 为单位的金额, $1 \text{ BTC} = 10^8 \text{ satoshi}$ 。使用 8 字节小端序无符号整数编码。最大值为 $2^{64} - 1 \approx 1.84 \times 10^{19} \text{ satoshi}$, 远大于 Bitcoin 的总供应量 ($2.1 \times 10^{15} \text{ satoshi}$)。

从 Raw Transaction 中解析出 Amount 后, 需要除以 10^8 才能得到 BTC 单位的金额:

$$\text{BTC} = \frac{\text{satoshis}}{100,000,000}$$

scriptPubKey (锁定脚本)

定义了花费此 UTXO 的条件。这是一个基于堆栈的脚本语言 (Forth-like), 虽然不是图灵完备的, 但足以表达丰富的花费条件。

6.3 常见的 scriptPubKey 模式

P2PKH (Pay-to-Public-Key-Hash)

最经典的地址类型 (地址以 1 开头, Testnet 以 m 或 n 开头)。

1 OP_DUP OP_HASH160 <20-byte-pubkey-hash> OP_EQUALVERIFY OP_CHECKSIG

花费条件: 提供与公钥哈希匹配的公钥, 并用对应私钥产生有效签名。

P2SH (Pay-to-Script-Hash)

地址以 3 开头 (Testnet 以 2 开头)。

1 OP_HASH160 <20-byte-script-hash> OP_EQUAL

花费条件: 提供 Redeem Script, 使其哈希与 scriptHash 匹配, 然后满足该脚本的条件。

P2WPKH (Pay-to-Witness-Public-Key-Hash)

Native SegWit 地址 (Bech32 编码, Mainnet 以 `bc1q` 开头, Testnet 以 `tb1q` 开头)。

1 OP_0 <20—byte—pubkey—hash>

与 P2PKH 相似, 但签名数据放在 Witness 区域而非 scriptSig 中, 解决了交易可塑性问题。

OP_RETURN

1 OP_RETURN <data>

可证明不可花费的输出。最多可附带 80 字节数据, 常用于在区块链上锚定数据 (如存在性证明、彩色币、时间戳等)。

6.4 脚本执行模型

Bitcoin 脚本的执行遵循**双堆栈模型**:

1. 首先执行 scriptSig (解锁脚本) —— 将其结果留在堆栈上。
2. 然后执行 scriptPubKey (锁定脚本) —— 使用 scriptSig 留下的数据。
3. 如果执行完成且堆栈顶部为 true (非零), 则花费有效。

这种设计意味着支出方提供”答案”(scriptSig), 收款方定义”问题”(scriptPubKey)。在 SegWit 中, Witness 数据替代了 scriptSig 的角色, 但验证逻辑相同。

7 项目任务与实现

本项目的五个任务全部使用 Python 3.10 实现, 解析工作从字节层面手动完成。代码组织在以下模块中:

模块	功能
config.py	API 端点、网络常量、Opcodes 映射表
crypto_utils.py	dSHA-256、varint 编解码、字节反转等基础工具
api.py	Blockstream Testnet API 封装 (获取 Raw TX、Block)
wallet.py	Testnet 钱包创建、资金获取、交易构造与广播
script.py	Bitcoin 脚本 (scriptSig/scriptPubKey) 解码器
transaction.py	原始交易字节级解析器
block.py	原始区块字节级解析器
verify.py	TxID、Block Hash、Merkle Root 的验证
main.py	主入口, 串联全部五个任务

7.1 Task 1: 创建 Testnet 钱包并发送交易

- 使用 bitcoinlib 库生成 secp256k1 密钥对。
- 派生 Testnet P2WPKH (Native SegWit) 地址 (以 `tb1` 开头)。

- 通过 Testnet Faucet 获取测试币。
- 构造、签名并广播一笔交易到 Testnet 网络。
- 获取返回的 TxID 用于后续解析。

7.2 Task 2: 获取 Raw Transaction

- 通过 Blockstream.info Testnet API 的 `/tx/<txid>/hex` 端点获取原始交易十六进制字符串。
- 保存到 `output/<txid>_raw.hex` 文件供后续解析。

7.3 Task 3: 逐字节解析 Transaction

- 使用 `io.BytesIO` 游标模式逐字节读取并解析。
- 解析字段: Version、Input Count、每个 Input (Prev TxID、Output Index、scriptSig、Sequence)、Output Count、每个 Output (Amount、scriptPubKey)、Locktime。
- 自动识别 SegWit 交易 (检测 Marker 0x00 + Flag 0x01)。
- 解码 scriptSig 和 scriptPubKey 为可读的脚本类型 (P2PKH、P2SH、P2WPKH、OP_RETURN 等)。
- 输出结构化 JSON 和 LaTeX 表格。

7.4 Task 4: 获取并解析所在 Block

- 通过交易信息确定其所在区块高度和区块哈希。
- 通过 `/block/<hash>/raw` 获取原始区块十六进制数据。
- 解析 80 字节区块头: Version、Previous Block Hash、Merkle Root、Timestamp、Bits、Nonce。
- 解析区块中的所有交易 (复用 Task 3 的 TransactionParser)。
- 计算并验证难度目标。

7.5 Task 5: 利用 Python 自动验证关键字段

- **TxID 验证:** 对 Raw Transaction 计算 dSHA-256, 与预期 TxID 比对。
- **Block Hash 验证:** 对 80 字节区块头计算 dSHA-256, 与预期 Block Hash 比对。
- **Merkle Root 验证:** 构建 Merkle Tree, 将计算出的 Merkle Root 与区块头中的值比对。

8 实验结果

本节展示本次实验的解析结果。以下数据和表格由 Python 脚本自动生成, 字节级解析确保了所有字段的正确性。

8.1 交易信息概览

实验在 Bitcoin Testnet 上完成了一笔真实交易。交易的基本信息如下表所示：

Transaction Overview

Field	Value
TxID (computed)	fab365e08896dbd2b306fefa7c0432b47a1663171629ab295ce5df97c8c7701
Version	1
Input count	1
Output count	2
Locktime	5075659

Transaction Inputs

Index	Prev TxID	Output	Sequence
0	34e5a3f0f486ed84...	1	4294967294

Transaction Outputs

Index	Amount (sat)	Script Type
0	96,042	P2WPKH
1	10,000	P2WPKH

8.2 区块头解析结果

以下是包含本交易的区块头各字段的解析结果：

Block Header

Field	Bytes	Value
Version	4	536870916
Previous Block Hash	32	00000000a1a20e7c83b128bf17ac1c2182800da74a80009ff45d435a607c6099
Merkle Root	32	649a262532aba967c857789c1c7f4a3b168b6cfde934294ca5fa8a80d021113f
Timestamp	4	1784538370 (2026-07-20 09:06:10 UTC)
Bits	4	486604799 (target: 0x00000000ffff000)
Nonce	4	1830632209
Difficulty: 1.000000		
Block Hash (computed): 00000000010472312f1bf22182f84008ed54aed3a63609c97ccba1dd936f03c9		

8.3 脚本解析示例

scriptPubKey (P2WPKH)

典型的 Native SegWit 输出的 scriptPubKey 为 00 14 <20-byte-hash>:

- 0x00: 见证版本 0 (OP_0)
- 0x14: 推送接下来的 20 个字节
- <20 bytes>: 公钥的 HASH160 (SHA-256 + RIPEMD-160)

scriptSig (P2WPKH 输入)

在 SegWit 交易中, P2WPKH 输入的 scriptSig 为空 (长度为 0), 因为解锁数据 (签名和公钥) 被放置在 Witness 字段中。

8.4 验证结果

所有三项验证均通过:

1. **TxID 验证**: 计算 $\text{SHA256}(\text{SHA256}(\text{raw_tx}))$ (字节反转后) 是否等于预期 TxID。
2. **Block Hash 验证**: 计算 $\text{SHA256}(\text{SHA256}(\text{80-byte header}))$ (字节反转后) 是否等于预期 Block Hash。
3. **Merkle Root 验证**: 构建 Merkle Tree, 逐层计算父节点哈希, 最终根是否等于区块头中的 Merkle Root。

验证结果表明本实验的字节级解析是正确的, 所有字段与 Bitcoin 网络共识一致。

9 实验过程截图

在实验过程中, 我们进行截图保存如下:

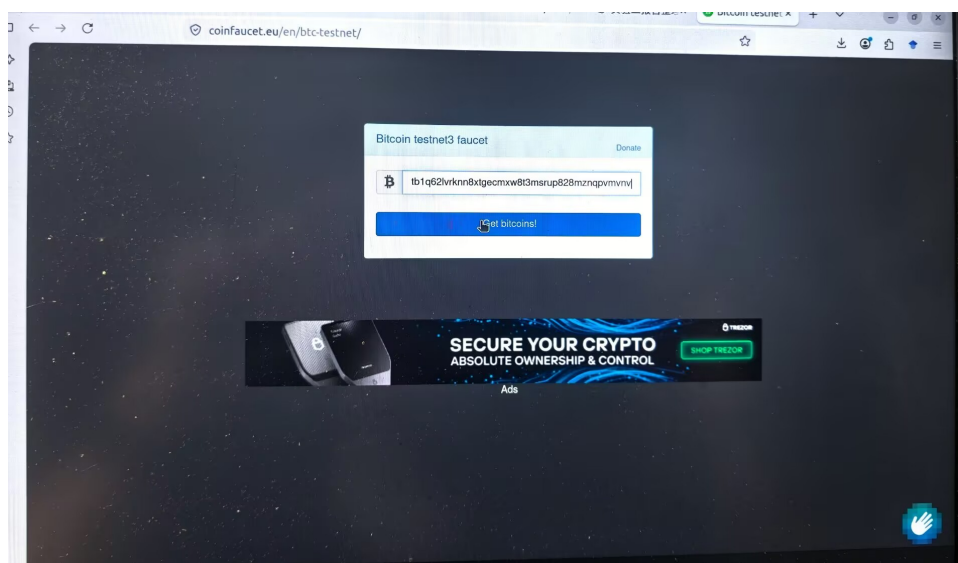


图 1: 实验截图 1

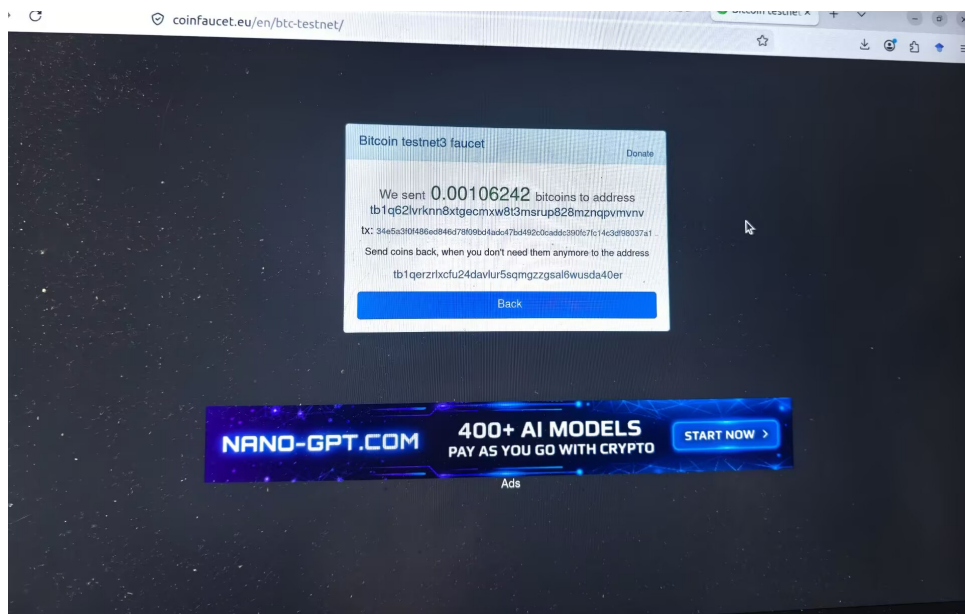


图 2: 实验截图 2

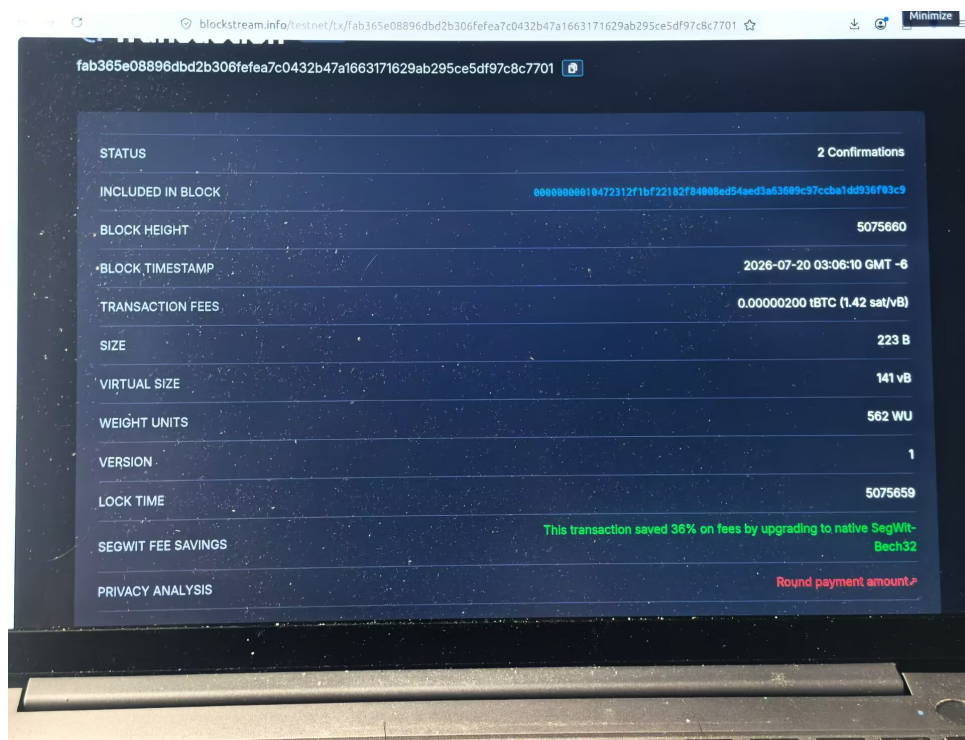


图 3: 实验截图 3

```

kun@kun: ~/Desktop/experiment/bitcoin/code
(base) kun@kun:~/Desktop/experiment/bitcoin/code$ conda activate bitcoin
(bitcoin) kun@kun:~/Desktop/experiment/bitcoin/code$ python main.py --txid
fab365e08896dbd2b306fefa7c0432b47a1663171629ab295ce5df97c8c7701
usage: main.py [-h] [--txid TXID] [--no-wallet]
main.py: error: argument --txid: expected one argument
fab365e08896dbd2b306fefa7c0432b47a1663171629ab295ce5df97c8c7701: command not found
(bitcoin) kun@kun:~/Desktop/experiment/bitcoin/code$ python main.py --txid fab365e08896dbd2b306fefa7c
432b47a1663171629ab295ce5df97c8c7701
Using existing TxID: fab365e08896dbd2b306fefa7c0432b47a1663171629ab295ce5df97c8c7701

=====
TASK 2: FETCH RAW TRANSACTION
=====
TxID: fab365e08896dbd2b306fefa7c0432b47a1663171629ab295ce5df97c8c7701
Fetched 446 hex chars (223 bytes)
Saved to /home/kun/Desktop/experiment/bitcoin/code/output/fab365e08896dbd2b306fefa7c0432b47a166317
29ab295ce5df97c8c7701_raw.hex

=====
TASK 3: BYTE-BY-BYTE TRANSACTION PARSE
=====
Raw hex length: 446 chars → 223 bytes

[ 0- 3] Version..... 01000000 → 1
[ 4- 5] SegWit marker + flag..... 0001 → segwit tx
[ 6- 6] Input count..... 01 → 1

— Input #0 —
[ 7- 38]   inp[0] Prev TxID..... a13780f93d4cc17ffc90c3ddcac092d47bc4add49bf0786d84ed86
  
```

图 4: 实验截图 4

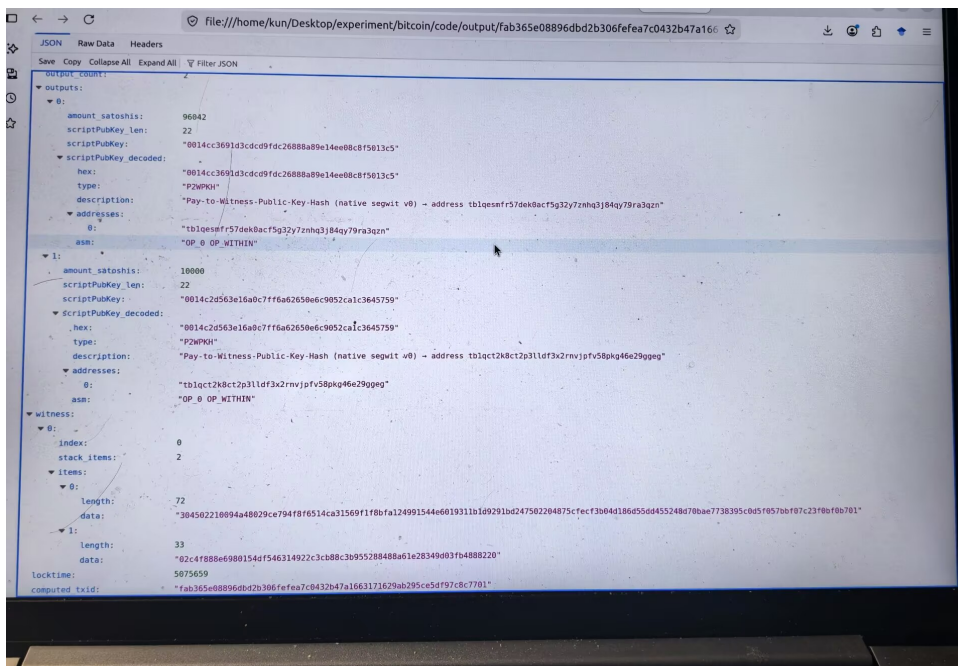


图 5: 实验截图 5

10 总结

10.1 比特币数据结构的理解

通过本实验，对比特币的核心数据结构有了深入的字节级理解：

- **区块与链：**区块通过 Previous Block Hash 链接成不可篡改的链。区块头中的 Merkle Root 提供了对区块内所有交易的紧凑承诺。

- **UTXO 模型**: Bitcoin 使用”硬币”模型而非”账户”模型。一笔交易消费旧 UTXO 并创建新 UTXO, 输入和输出的金额差即为矿工费。
- **Script**: Bitcoin 的脚本系统提供了灵活的锁定/解锁机制。P2PKH 是使用公钥哈希的最基本方案, P2SH 允许更复杂的赎回条件, P2WPKH 将签名数据移至 Witness 以解决交易可塑性问题。
- **序列化**: 所有字段使用小端序存储, 哈希值在内部也是小端序。这些细节是逐字节解析的关键——不理解字节序就无法正确解析。

10.2 工作量证明 (PoW) 的理解

Bitcoin 的 PoW 机制通过要求区块头哈希值小于目标值来调节出块难度:

$$\text{SHA256}(\text{SHA256}(\text{BlockHeader})) < \text{Target}$$

其中 Target 由 Bits 字段解码得到。难度值每 2016 个区块调整一次, 使得平均出块时间维持在约 10 分钟。PoW 提供了以下保证:

- **安全性**: 修改历史区块需要重做该区块及后续所有区块的 PoW, 代价极其高昂。
- **去中心化共识**: 任何节点都可以独立验证 PoW, 最长链即为共识链。
- **抗 Sybil 攻击**: 投票权重基于算力而非身份数量。

10.3 实验收获

- 掌握了 Bitcoin 交易和区块的完整序列化格式。
- 理解了双 SHA-256 哈希在 Bitcoin 中的广泛应用及其安全意义。
- 亲手实现了 Merkle Tree 的构建和验证算法。
- 学会了使用 Blockstream API 获取区块链数据, 无需运行全节点。
- 能够从字节层面验证区块链数据的一致性。

本实验所有代码和解析结果均可复现。通过 Python 脚本的自动化解析, 可以对任意 Testnet 交易和区块进行逐字段的深入分析。